Mini Project Title:

"Smart Home Device Controller"

📄 Problem Definition:

In a smart home setup, different types of electronic devices (like TVs, Lights, Fans, and Speakers) respond to common actions such as turning **on**, turning **off**, and adjusting **settings**. However, each device responds differently to these actions.

Create a Java-based system that simulates a **unified interface for controlling various smart devices**, ensuring **loose coupling and polymorphism** through the use of **interfaces only**.

🎯 Objective:

To design a system where all smart devices follow a **common interface (`Controllable`)** but implement their own behaviour for `turnOn()`, `turnOff()`, and `setMode()`.

Requirements:

1. Define an Interface: `Controllable`

```
interface Controllable {
    void turnOn();
    void turnOff();
    void setMode(String mode);
}
```

---

2. Implementing Classes (Devices):

- `Light`
- `Fan`
- `TV`
- `Speaker`

Each class must:

- Implement `Controllable`
- Customize `setMode()` method for its own setting (e.g., Light: "Dim", Fan: "High", TV: "HD", Speaker: "Bass")

---

3. Main Controller Class (Simulation)

- Create an array of `Controllable` devices.
- Use a loop to **turn all devices on**, **set mode**, and **turn off**.
- Show how interface references enable flexibility and polymorphism.

□ Sample Output:

```
Light turned ON
Light mode set to: Dim
Light turned OFF

Fan turned ON
Fan mode set to: High
Fan turned OFF

TV turned ON
TV mode set to: HD
TV turned OFF

Speaker turned ON
Speaker mode set to: Bass
Speaker turned OFF
```

1. Banking Domain

Problem:
Design a banking system using an interface `AccountOperations` with methods: `deposit()`, `withdraw()`, and `checkBalance()`.
Create classes `SavingsAccount`, `CurrentAccount`, and `LoanAccount` that implement these methods with different logic (e.g., savings has withdrawal limits, loan accounts do not allow deposits).
Use polymorphism to simulate multiple accounts.

---

2. E-commerce Domain

Problem:
Define an interface `PaymentGateway` with methods: `pay(double amount)` and `refund(double amount)`.
Implement classes `CreditCard`, `UPI`, and `NetBanking` that simulate payment processing differently.
Create a `Checkout` class that uses a `PaymentGateway` reference to complete transactions dynamically.

---

3. Transportation/Vehicle System

Problem:
Create an interface `VehicleControl` with methods: `start()`, `stop()`, and `changeGear(int gear)`.
Implement `Car`, `Bike`, and `Truck` classes, each with custom implementations.
Add a test class that controls different vehicles using interface references and loops.

---

4. Education/Exam System

Problem:
Define an interface `ExamEvaluator` with methods: `evaluateMarks()`, `calculateGrade()`.
Implement for different types of exams: `TheoryExam`, `PracticalExam`, and `OnlineQuiz`.
Each class uses different grading logic. Write a class that prints the evaluated result using interface reference.